



Fall 2026

Challenge Week Materials

University of Maryland, College Park

—

Phase II

Technology Challenge

Phase II

Congratulations on making it to the second phase of UMD Loop's Challenge Week! It is an honor to participate in the following challenge, and through any outcome, you will almost certainly learn skills applicable to your major and your eventual career.

Logistics:

The second phase of Challenge Week begins today, Wednesday, 9/16, and your deliverables will be due by 2:00 a.m. on Tuesday, 9/22. From Tuesday until Friday, 9/25, each team will find a time slot to present to UMD Loop leadership. Decisions will be announced at **7:00 p.m.** on Saturday, 9/26. The first GBM, which is **mandatory** for all members, will be **4:00 p.m.** on Sunday, 9/27.

Open Lab Hours:

As a reminder, *everyone* is encouraged to ask any questions that you may have throughout the process. In order to ease this process, members of the UMD Loop Executive Team and Leadership Team - as well as current members - will be in IPST during [these Open Lab Hours](#). Please use this time to come in, work on your challenge with your group, and ask questions. The more time *we* see *you* doing work, the better idea we will have of you and your work ethic.

Deliverable:

Your final deliverable will be a 15-30 minute group presentation on a medium of your choosing. Presentations should be professional, and presented as such; they should detail your work, your thought processes, how you all worked together as a group, and your final product. The presentations will be in person, but should be submitted to [this form](#) no later than **2:00 a.m. on Tuesday, September 22nd**.

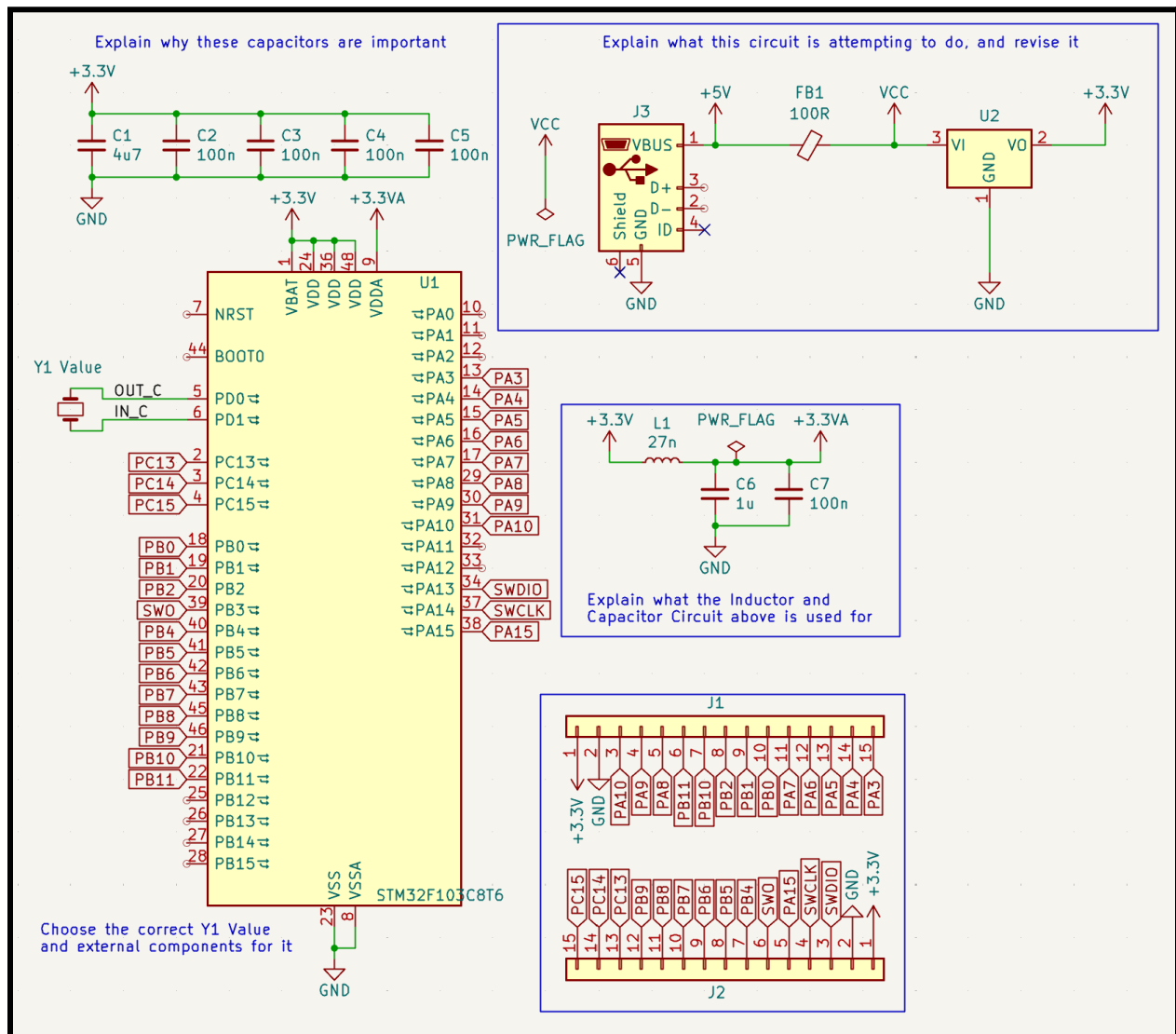
Good luck!

Challenges:

E0. KiCAD MCU Circuit Revision

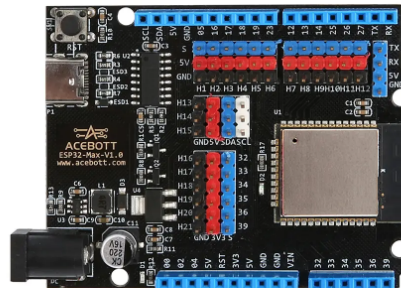
Given the following KiCAD Circuit diagram, identify its purpose and **AS MANY** areas for improvement (one CLEAR issue is there being no method of programming the MCU).

It is highly recommended that you correct the circuit before moving below onto external functionality (E1.). Software depends on you for not messing up the creation of PCBs, which could cost literal hours of debugging with a voltmeter.



E1. Basic Data Conversion & Sensor Manipulation

Assume the circuit you designed above works as intended. Now you are tasked with programming your circuit! Because there isn't an abundance of STM32 Dev Boards, ESP32s will be provided for you to program (an image of one is below). You are allowed to use the ARDUINO IDE, but be expected to be asked questions regarding what is abstracted when you are programming. You will be programming the following systems below. Each system must be able to run on one board only. If your team does not have additional equipment required for each system, you are also allowed to capture ideal **input and output signals** and explain how they enable your system to work.



- **System ONE:** turns on a Red, Green, or Blue LED depending on how close an inputted HEX value is. This Hex value is received through any pin on the ESP32, and you will decide on how your RGB LED(s) will light up
- **System TWO:** controls a typical B2122 servo found on Adafruit. The servo rotates to a specific degree depending on what information you send through the ESP32
- **System THREE:** sends an analog signal to play a tone for a set duration. The first piece of information you send is the duration. This is open ended, so you could increase or decrease a specific tone's duration based on the read voltage
- **System FOUR:** receives data from an external signal. The signal may range from a number between 0 and 255 to This signal is of your own choice, but it must be of the following communication protocols: CAN, I2C, or USART (USB)

E2. Embedded Engineering

A complaint has been received by Neil (the embedded software lead) and Asgar (an electrical member) involving organizational issues with the systems you have made above. When a single microcontroller is handling multiple tasks, such as communicating over CAN, reading sensors, controlling motors, etc., it is important to manage these tasks efficiently so as to not cause delays or miss critical information.

Some methods of solving this multitasking problem include using an RTOS (Real Time Operating System). One in particular that we are beginning to use is Zephyr RTOS which comes with the benefit of code being modular and reusable. Your task is to work with Software and create Zephyr device drivers to control the hardware used above (bindings files, instantiation, API, etc.). These drivers should be in the form of Zephyr modules. With these modules, write an application that uses them to complete the E1. challenge again and show off your hard work! This is quite open ended; many of the major decisions are left up to you. We do not have all or any answers to some questions you may ask; we want you to explore this section, and see how well you can solve the issue of (how can I develop modular code and have systems work together coherently?)

Deliverables:

For Part 0, you must **detail & present** the revisions you made onto the existing schematic. It must be clear how you will **safely power** the circuit, how you will **properly program** the circuit, and how you will **purchase** the circuit (provide a Bill of Materials).

For Part 1, you will explain your process for programming **each system**. Mentioning all areas which you recognize the limitations of programming in Arduino IDE if you choose to program the ESP32 with it.

For Part 2, you must **create a block diagram** showing all important directories in your zephyr project. With this block diagram, present how you coordinated with Software on how to tackle this task. If you were successful in creating device drivers, showcase them in your presentation.

S1: Navigation PyQT GUI

Build on your Phase 1 navigation stack by creating a custom PyQt command-and-control GUI for your simulated rover. The challenge is based on the autonomous navigation requirements of the 2026 University Rover Challenge.

The GUI must be your own implementation. RViz and terminal commands may be used for debugging, but the final mission must be operated entirely through the GUI.

Remember this is designed to be an *Impossible* challenge. Even if you barely finish a portion of the required functionality, you can still have a great presentation if you are able to adequately explain what you learned. Quality over quantity applies a lot here.

Required Functionality:

Your system must

- Accept WGS 84 coordinates
- Convert between WGS 84 coordinates and a local REP 103-compatible ROS coordinate frame.
- Associate the supplied starting coordinate with the rover's spawn position.
- Send the rover autonomously to a supplied destination coordinate.
- Display an offline map of the Mars Desert Research Station area.
- Display the rover's position, heading, traveled path, destination, and current mission state on the map.
- Allow targets to be created, edited, reordered, saved, and loaded.
- Support just GNSS target type in the interface. No need for object/aruco detection here. Actual perception may be simulated.
- Start, monitor, abort, stop, and reset a mission through the GUI.
- Display stale, missing, or disconnected ROS data appropriately.
- Record mission events, operator commands, target results, and coordinate conversions in an exportable log.
- Navigate around at least one obstacle using your Phase 1 work.

The Gazebo world does not need to resemble MDRS (but would be nice). It only needs to support navigation and obstacle avoidance. The geographic alignment

applies to the coordinate system and offline GUI map. The PyQt gui doesn't have to be pretty here. We're looking for functionality.

ROS Requirements:

Your solution should demonstrate proper use of:

- ROS topics, services, and/or actions
- Coordinate transforms
- Appropriate frame IDs
- ROS timestamps
- QoS settings
- DDS-based communication
- Separation between the GUI, mission management, and rover control
- Safe behavior when nodes, telemetry, or communications become unavailable

Assume the real rover-to-base-station connection may be **unreliable** and bandwidth-limited. You do not need to simulate these network conditions, but your design should account for them.

Repository and Presentation Deliverables:

Push all code, launch files, map assets, simulation files, and setup instructions to a public GitHub repository.

In your engineering log, explain:

- How coordinate conversion works
- How the map and ROS frames are aligned
- How the GUI communicates with ROS
- How DDS and QoS affect your design
- What happens during stale data, lost communication, or node failure
- How aborts and unreachable targets are handled
- What would need to change for real rover hardware and radio operation
- What remains unfinished
- The GUI and simulation launching
- Offline map operation without internet access

- Coordinate entry in all three required formats
- Rejection of an invalid coordinate
- Conversion between geographic and local ROS coordinates
- Mission creation through the GUI
- Autonomous travel from the supplied start coordinate to the supplied destination
- Obstacle avoidance
- Live mission and rover status
- Successful arrival within a documented tolerance
- An abort, unreachable target, lost-data, or other failure case
- Exported mission logs
- Replace Fast DDS with a ROS 2 Jazzy-compatible Zenoh setup. Demonstrate the system operating through Zenoh and compare its behavior and configuration with your original DDS setup.
- Use the Nav2 framework for your navigation stack. Implement localization, more sensors, global/local planning, etc.
- Simulate the Utah landscape around the MDRS (Mars desert research station) in Gazebo. USGS is a great resource to pull elevation maps to convert to occupancy grids.

Your presentation should not be longer than 20 mins.

Review the [official URC 2026 requirements](#) and [URC 2026 Q&A](#). You are responsible for identifying which autonomous-navigation rules and field constraints should influence your design.

Note:

AI use for code is encouraged here and you should be able to explain how you used AI in the coding process.

****IMPORTANT****

This does not mean that you can have code in the repository that you cannot explain. We will be grilling you during presentations so be ready to explain every part of your system. AI is a tool, not something to rely completely on you won't have access to it during competition so KNOW YOUR STUFF.

S2: Autotyping Challenge

Build a ROS 2 node that autonomously types a launch key on a simulated keyboard. The challenge is based on the autonomous typing task of the 2026 University Rover Challenge.

The simulator is provided: [AutoTypingChallengeSim](#). It gives you a five-joint, velocity-controlled arm with a forearm camera, facing a keyboard on a panel marked with ArUco markers. Start with docs/INFO.md for setup; docs/INTERFACES.md is the complete reference.

Your node must be your own implementation and may interact with the simulator only through the ROS topics and services in INTERFACES.md. The dashboard, teleop and debug_press_feedback may be used for debugging, but the final attempt must run fully autonomously with teleop and debug_press_feedback off.

Remember this is designed to be an *Impossible* challenge. Even if you barely finish a portion of the required functionality, you can still have a great presentation if you are able to adequately explain what you learned. Quality over quantity applies a lot here.

Required Functionality

Your system must:

- Read the launch key from /sim/launch_key.
- Control the arm in closed loop from /joint_states, within its joint and velocity limits.
- Determine the panel's position and orientation from the camera image.
- Determine where the keys of the launch key are. Key positions are not provided.
- Move the arm to a pose from which every character of the launch key can be pressed.
- Aim at and press each character in order, then publish /sim/done.

- Type the launch key exactly, so /sim/result reports an exact match.
- Work on any episode seed without code changes. The panel's placement changes between episodes.
- Record detections, pose estimates, commands and press decisions in an exportable log.

ROS Requirements

Your solution should demonstrate proper use of:

- ROS topics and services
- Coordinate transforms
- Appropriate frame IDs
- ROS timestamps
- QoS settings
- DDS-based communication
- Separation between perception, planning and arm control
- Safe behavior when nodes, joint states or camera data become unavailable

Assume real camera data and arm motion are less ideal than the simulator's. You do not need to simulate this, but your design should account for it.

Repository, Log, and Presentation

Push all code, launch files and setup instructions to a public GitHub repository.

In your presentation, explain:

- How the panel's position and orientation are estimated, and how accurate the estimate is
- How key positions are determined
- How the camera, arm and panel frames are related
- How the typing pose is chosen
- How the arm is controlled, and how your node decides it is safe to press
- What happens during stale data or node failure
- What would need to change for real rover hardware

- What remains unfinished
- Type the launch key correctly on 10 consecutive episodes, advancing with /sim/reset, without restarting or retuning your node.
- Write a URDF for the arm from the numbers in INTERFACES.md and use MoveIt 2 to plan arm motion. Execute its plans through the velocity interface.
- Minimize the time from launch to /sim/done while keeping an exact match. Report your elapsed times across several seeds.
- The simulator and your node launching from your documented setup
- Marker detection and your panel estimate, shown in your own visualization
- The arm moving to a pose from which the launch key is reachable
- The full launch key typed autonomously, with the dashboard's result panel showing the outcome
- The same code succeeding on a second seed you did not tune for
- A failure case: lost camera data, an unreachable key, a rejected press, or another fault
- Exported logs

Your presentation should not be longer than 20 mins.

Review the [official URC 2026 requirements](#) and [URC 2026 Q&A](#). You are responsible for identifying which autonomous-typing rules and field constraints should influence your design.

Note:

AI use for code is encouraged here and you should be able to explain how you used AI in the coding process.

****IMPORTANT****

This does not mean that you can have code in the repository that you cannot explain. We will be grilling you during presentations so be ready to explain every part of your system. AI is a tool, not something to rely completely on you won't have access to it during competition so KNOW YOUR STUFF.